SGit vaults in 90 seconds

Encrypted, versioned folders the server can never read

2026-09-01

made by a Claude Code agent, unattended

13 scenes · 02:05 landscape

source: reel.json

00:00 OPENING 9.9 S

What this video covers

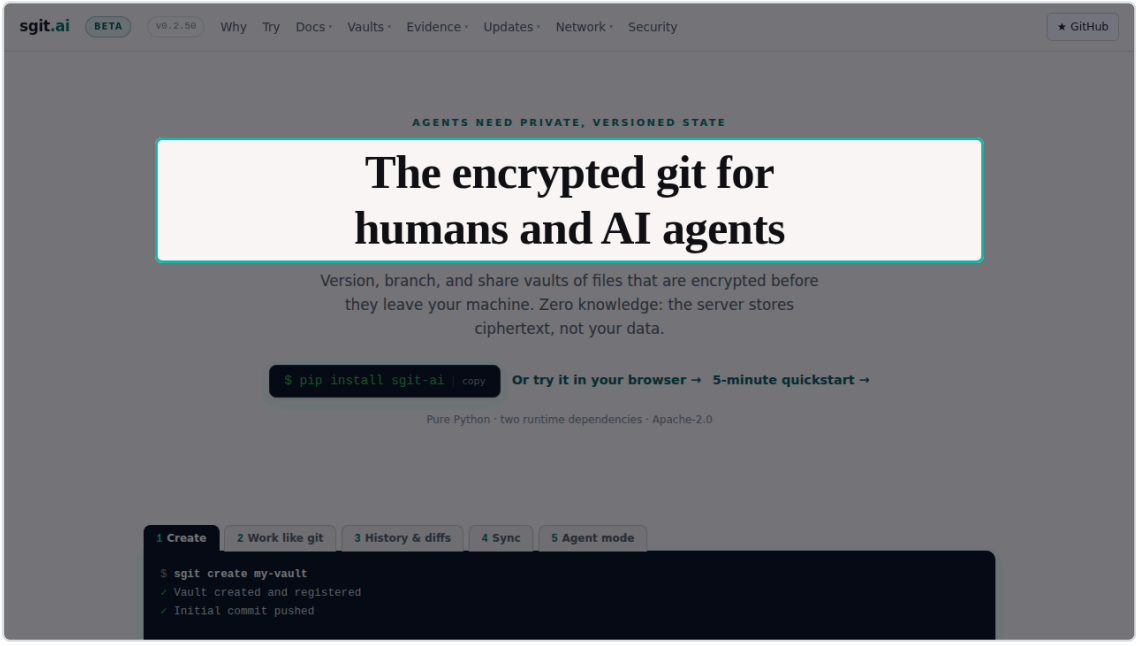
> In the next ninety seconds: what a vault is, what the server can and cannot see, and a published vault opened live in the browser.

00:09 SCENE 1 OF 13 S01 7.9 S

Encrypted before it leaves

> This is sgit. Git workflows over folders that are encrypted before anything leaves your machine.

still · https://sgit.ai/ · spot heading:The encrypted git

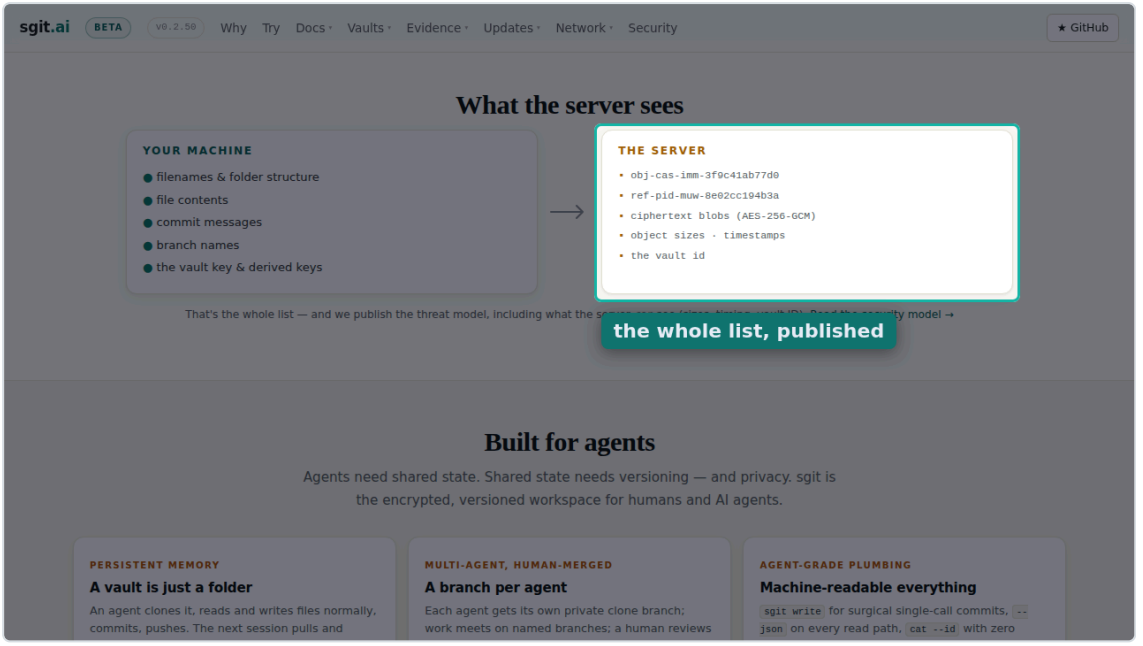


00:17 SCENE 2 OF 13 S02 9.2 S

The server sees ciphertext

> The server stores ciphertext under opaque identifiers. File names, contents and commit messages never reach it.

still · https://sgit.ai/ · scroll to “What the server sees” · spot right-column · label “the whole list, published”

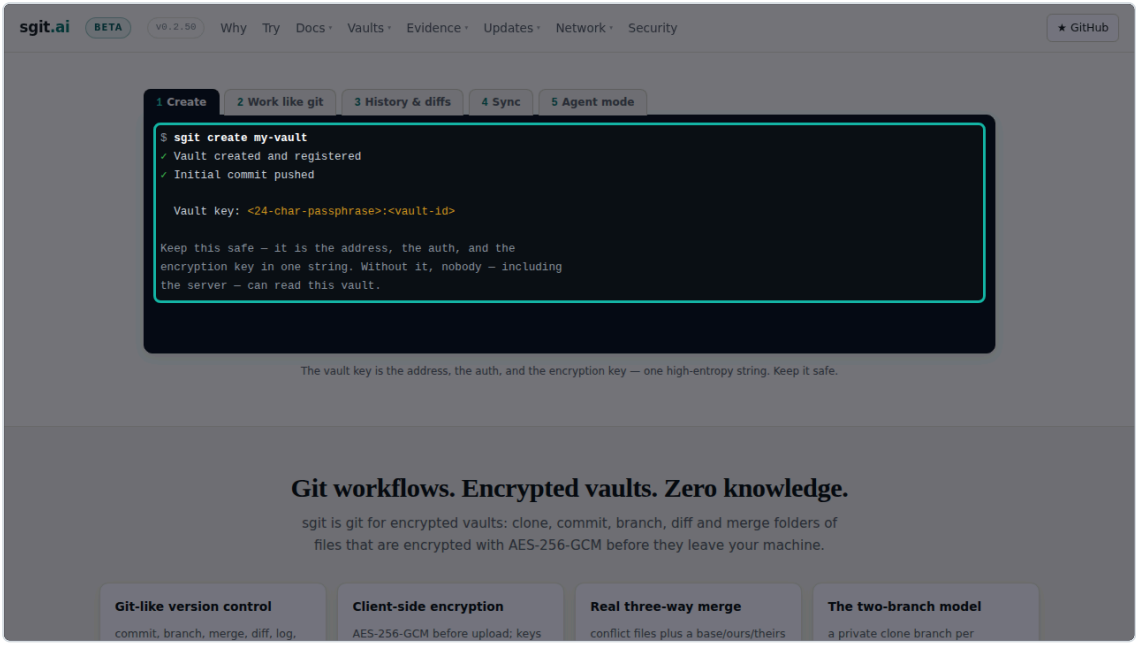


00:27 SCENE 3 OF 13 S03 2.9 S

One command

> Creating a vault is one command.

still · https://sgit.ai/ · scroll to “Work like git” · spot terminal

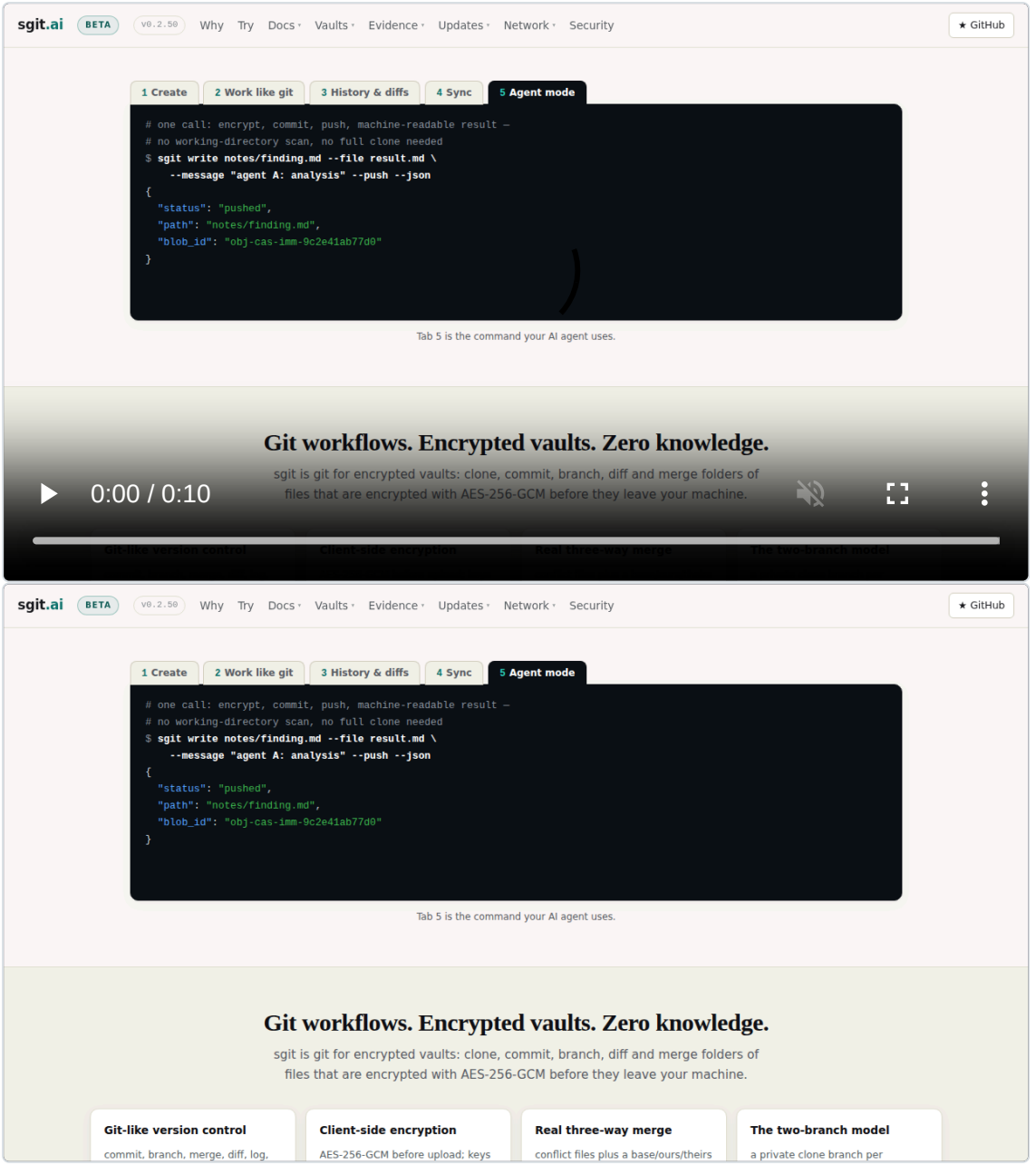


00:29 SCENE 4 OF 13 S04 7.6 S

Work like git

> Then you work like git. Status, commit, history, sync, and an agent mode built for machines.

clip · https://sgit.ai/ · scroll to “Work like git”

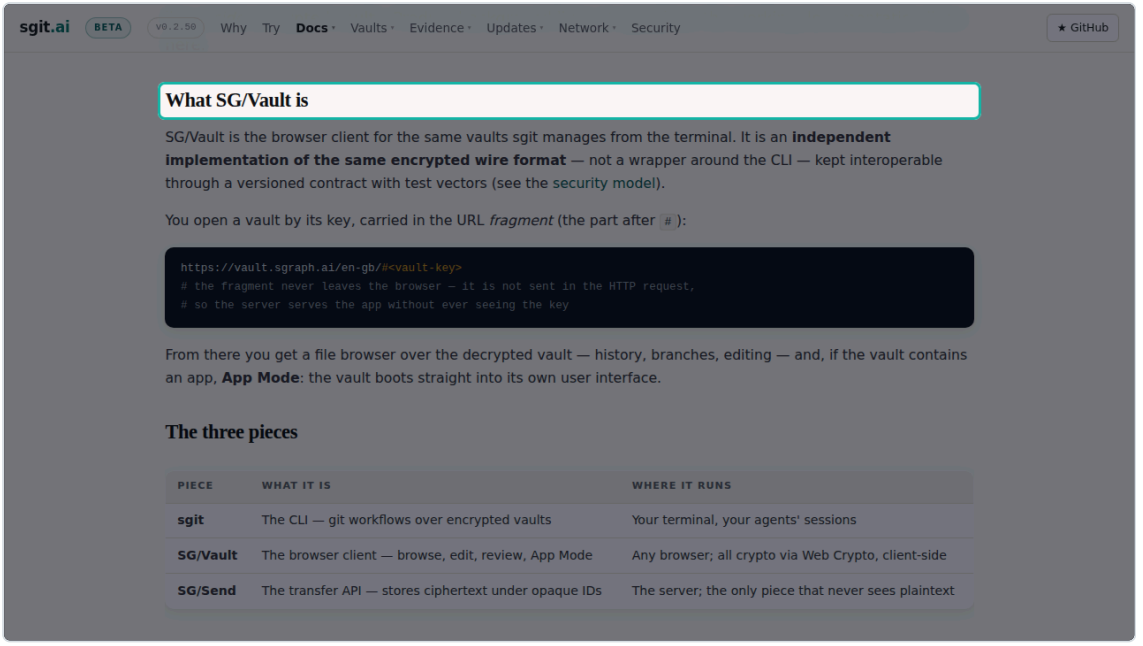


00:37 SCENE 5 OF 13 S05 8.8 S

Same vault, in the browser

> SG Vault is the same vault in the browser. One encrypted wire format, no wrapper around the command line.

still · https://sgit.ai/vault/ · scroll to “What SG/Vault is” · spot heading:What SG/Vault is

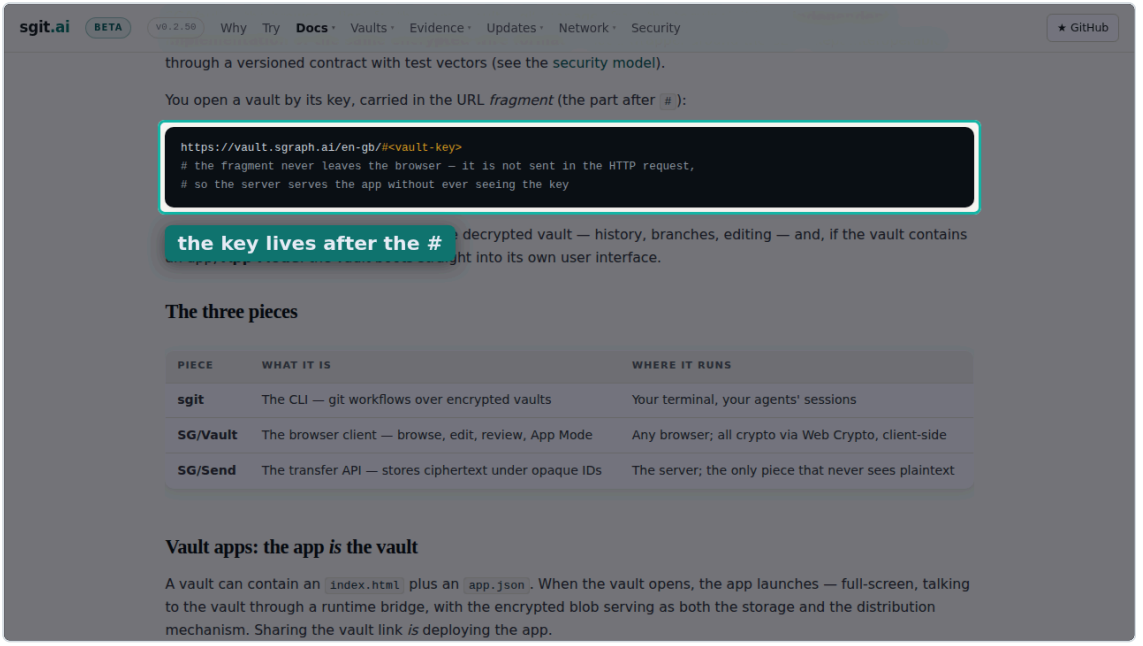


00:46 SCENE 6 OF 13 S06 9.5 S

The key never leaves the browser

> You open a vault by its key, carried in the URL fragment. The fragment is never sent, so the server never sees the key.

still · https://sgit.ai/vault/ · scroll to “You open a vault by its key” · spot code · label “the key lives after the #”

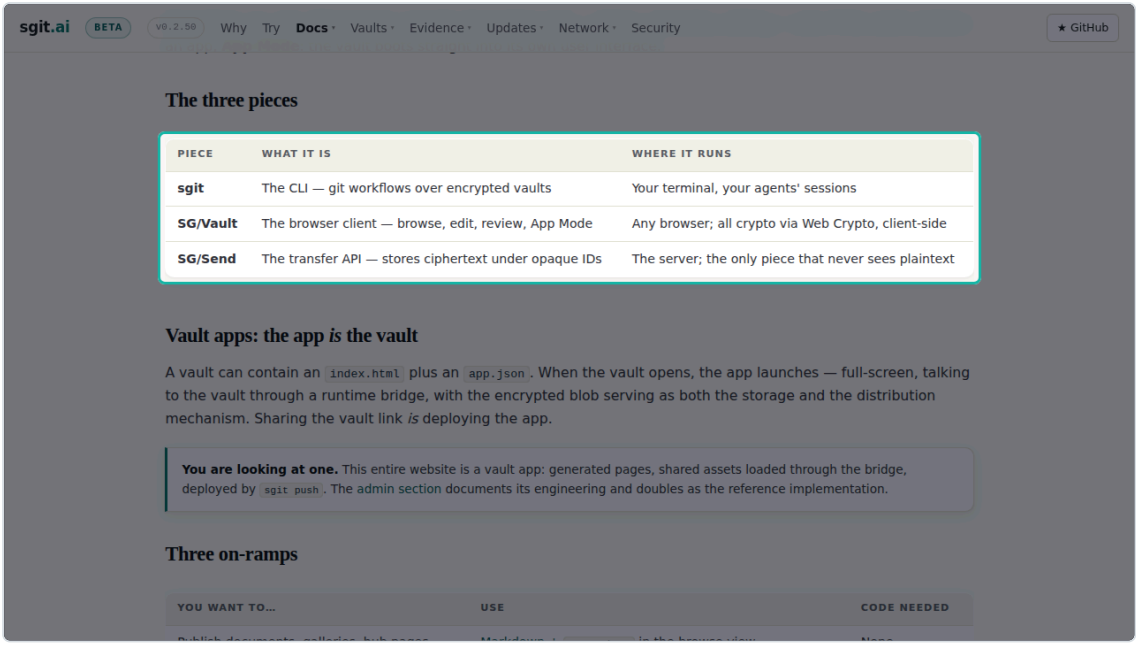


00:55 SCENE 7 OF 13 S07 12.9 S

Three pieces, one wire format

> Three pieces. The sgit command line, the SG Vault browser client, and SG Send, the transfer API that never sees plaintext.

still · https://sgit.ai/vault/ · scroll to “The three pieces” · spot table

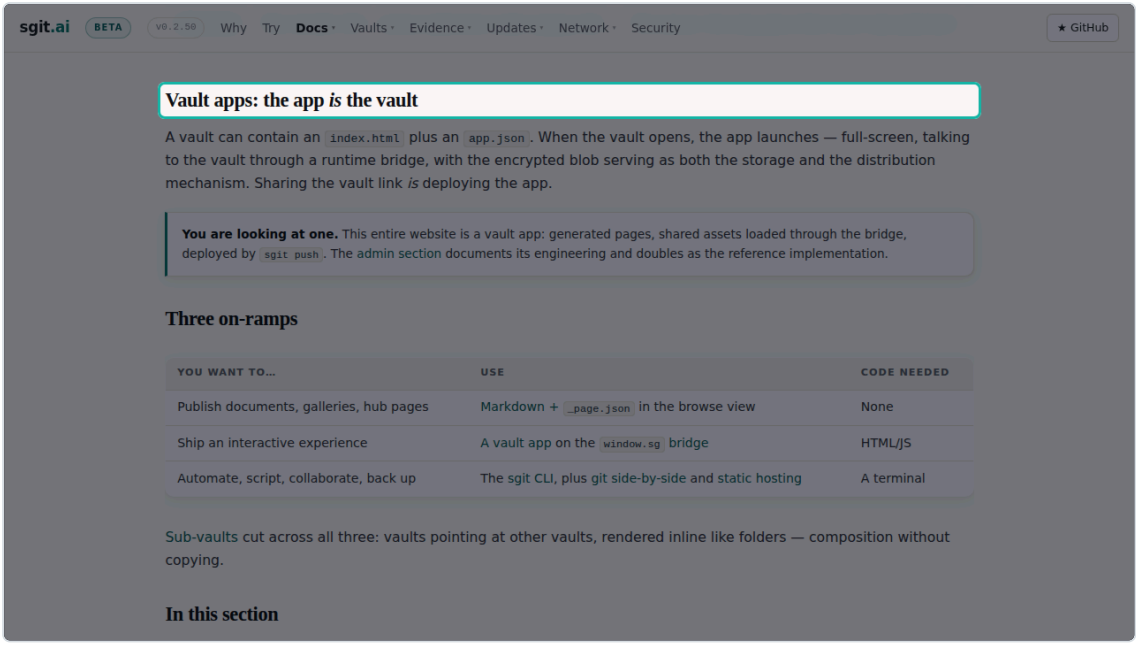


01:08 SCENE 8 OF 13 S08 5.7 S

The app is the vault

> A vault can contain an app. Sharing the vault link is deploying the app.

still · https://sgit.ai/vault/ · scroll to “Vault apps: the app is the vault” · spot heading:Vault apps: the app is the vault

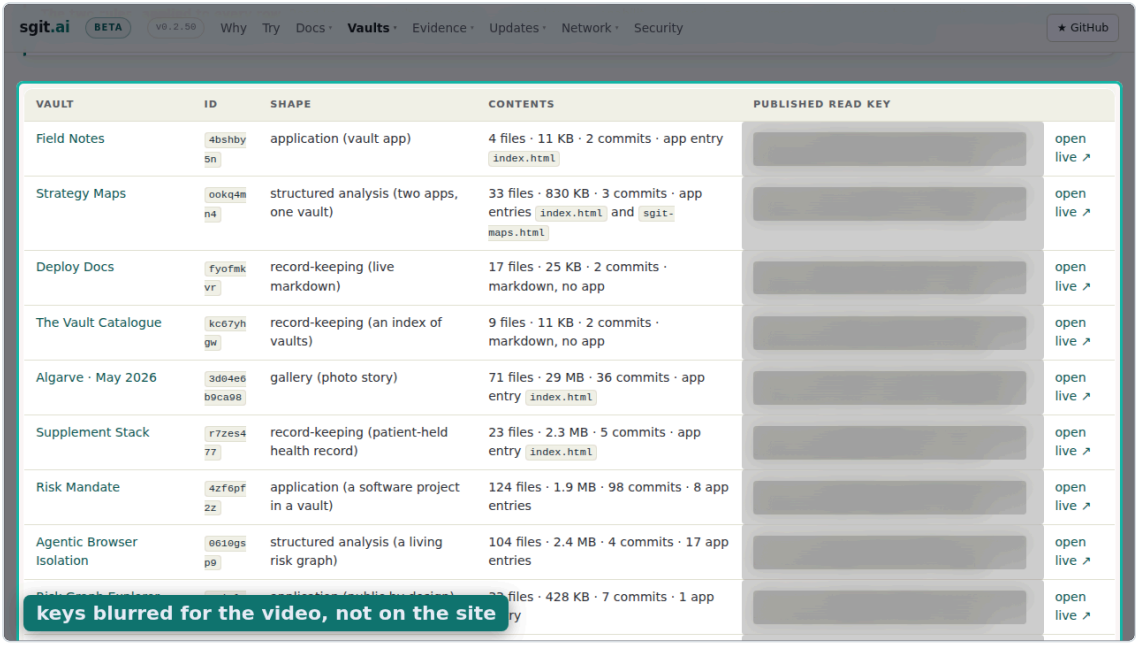


01:14 SCENE 9 OF 13 S09 9.0 S

Read keys, published on purpose

> sgit dot ai publishes read keys for its demo vaults on purpose. A read key cannot become write access.

still · https://sgit.ai/demos/vaults/index.html · scroll to “Published read key” · spot table · blur keys · label “keys blurred for the video, not on the site”

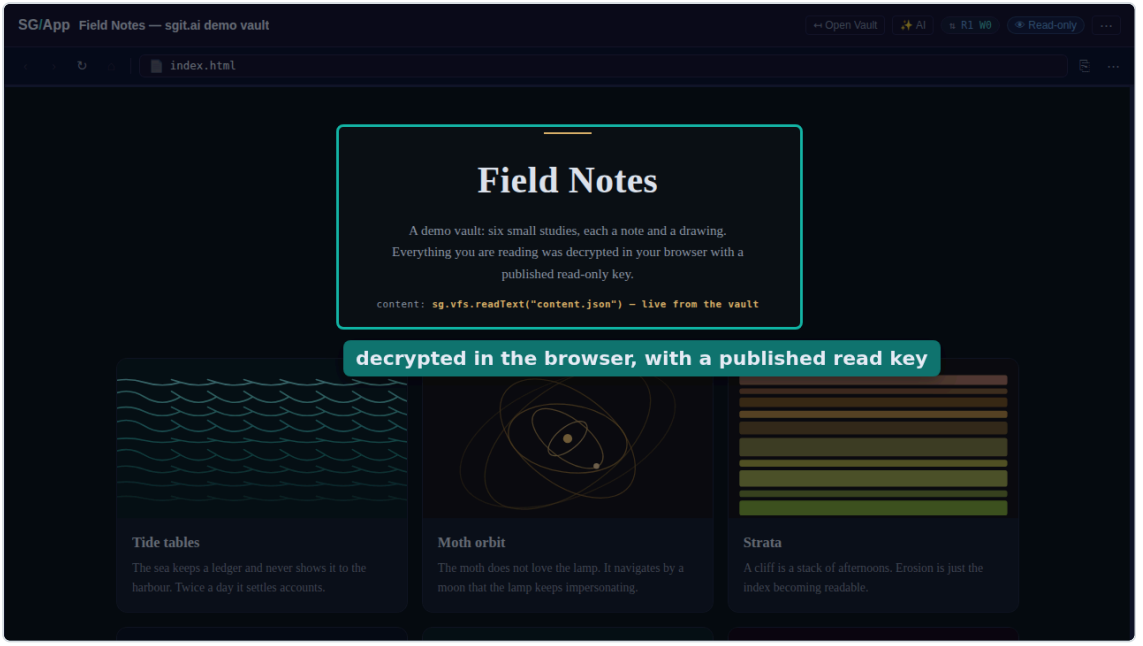


01:23 SCENE 10 OF 13 S10 9.1 S

Decrypted in your browser

> Here is one, opened live. Everything on this page was decrypted in the browser with a published read only key.

still · published vault “Field Notes” · spot rect · label “decrypted in the browser, with a published read key”

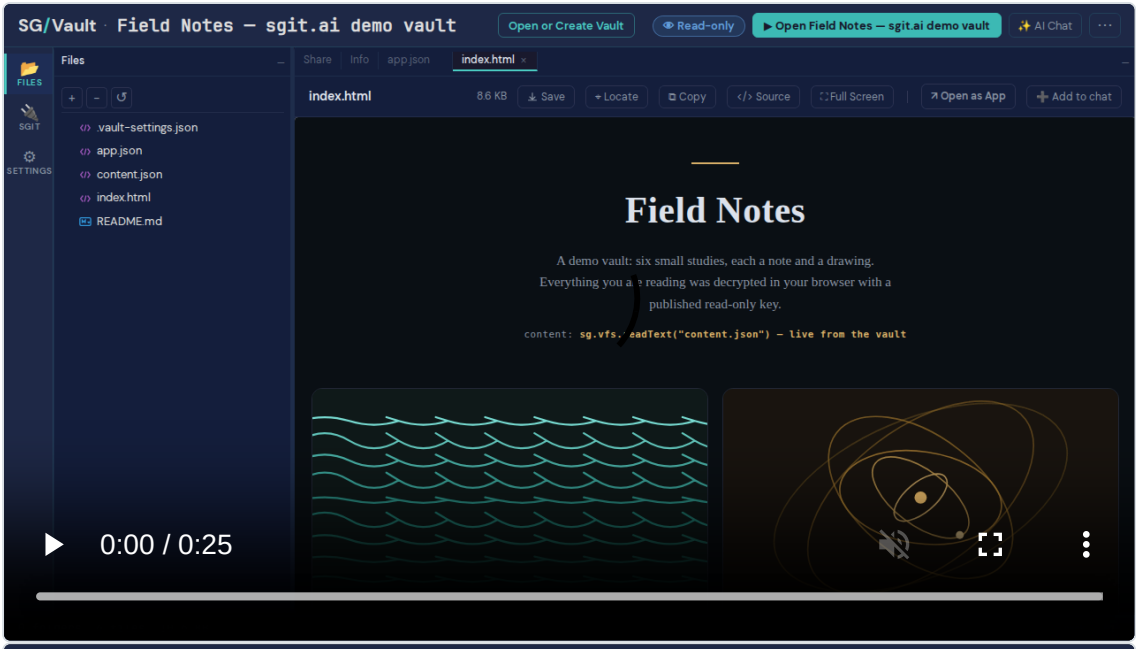


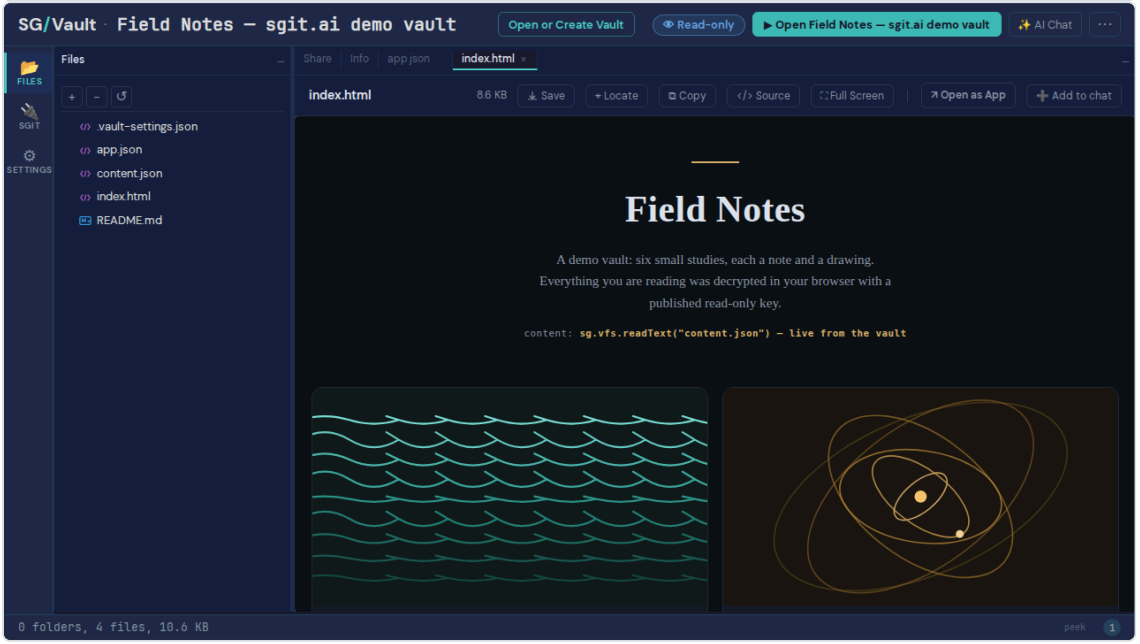
01:32 SCENE 11 OF 13 S11 9.0 S

Files, history, branches

> Behind the app is the vault itself. Files, history and branches, in the same file browser you get for any vault.

clip · published vault “Field Notes”



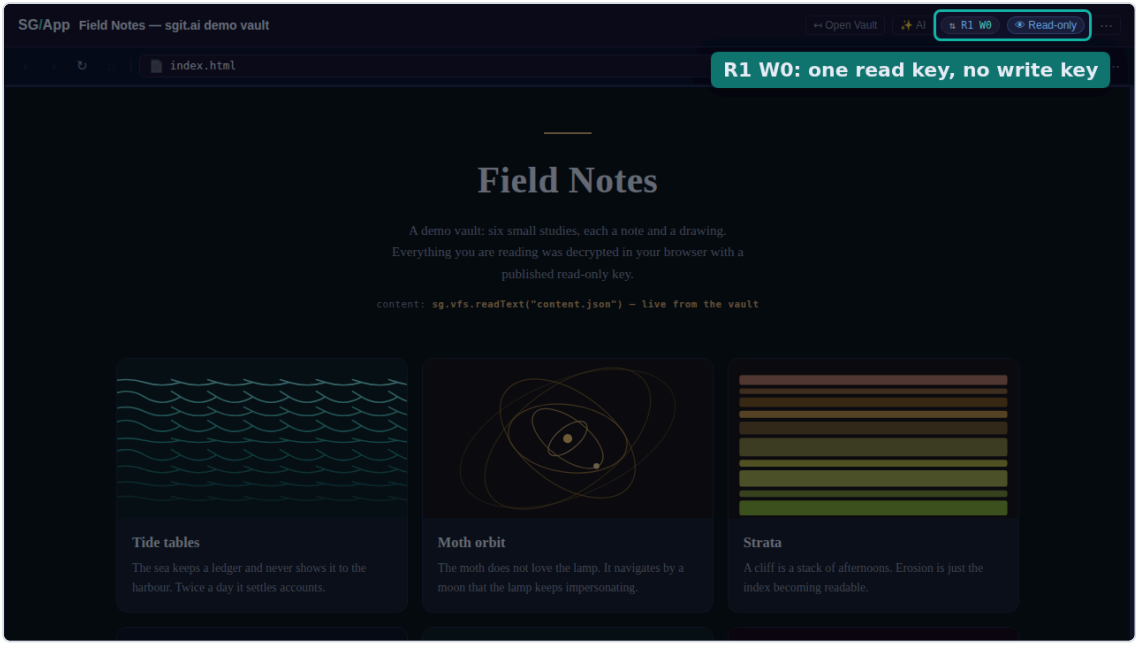


01:41 SCENE 12 OF 13 S12 5.3 S

Read-only by construction

> And the badge says what the key allows. One read, zero writes.

still · published vault “Field Notes” · spot badges · label “R1 W0: one read key, no write key”

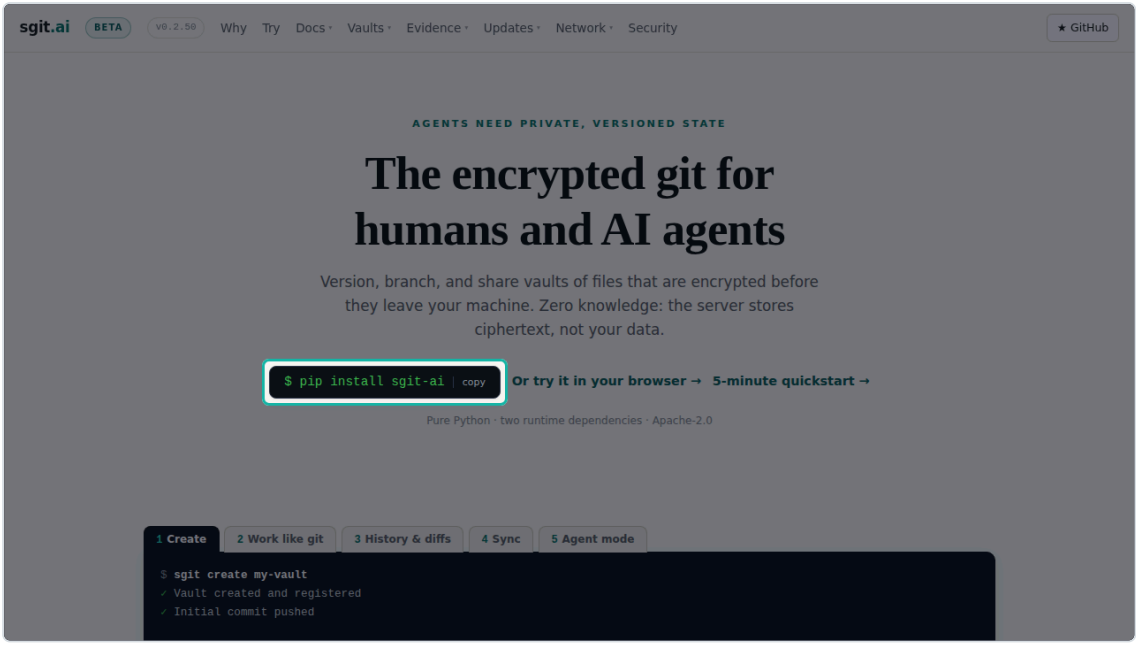


01:46 SCENE 13 OF 13 S13 7.3 S

pip install sgit-ai

> pip install sgit dash ai. Or try it in your browser, at sgit dot ai.

still · https://sgit.ai/ · spot pip



01:54 CLOSING 11.6 S

How this video was made

> This video was made by an agent, end to end, with no human at a screen and no API cost. The numbers are on the screen.

MADE BY a Claude Code agent, unattended

WHEN 2026-09-01

SCRIPT 13 scenes + title and closing · 269 words · reel.json written first

SCREENSHOTS 15 shot, 13 used · Playwright, headless · 93 s

NARRATION Kokoro-82M in the browser (2 workers), voice am_michael · 125.7 s of speech in 166 s · model load 7.6 s

RENDER video-creator, canvas + MediaRecorder, real time · 126 s

PIPELINE WALL CLOCK 394 s: capture 93 + compose and narrate 176 + render 126

API COST $0.00 · no LLM, TTS or image API calls

COMPUTE one 4-core container · agent session tokens not metered here

Transcript

The narration in order, as plain text, ready to paste next to the video.

In the next ninety seconds: what a vault is, what the server can and cannot see, and a published vault opened live in the browser.

This is sgit. Git workflows over folders that are encrypted before anything leaves your machine.

The server stores ciphertext under opaque identifiers. File names, contents and commit messages never reach it.

Creating a vault is one command.

Then you work like git. Status, commit, history, sync, and an agent mode built for machines.

SG Vault is the same vault in the browser. One encrypted wire format, no wrapper around the command line.

You open a vault by its key, carried in the URL fragment. The fragment is never sent, so the server never sees the key.

Three pieces. The sgit command line, the SG Vault browser client, and SG Send, the transfer API that never sees plaintext.

A vault can contain an app. Sharing the vault link is deploying the app.

sgit dot ai publishes read keys for its demo vaults on purpose. A read key cannot become write access.

Here is one, opened live. Everything on this page was decrypted in the browser with a published read only key.

Behind the app is the vault itself. Files, history and branches, in the same file browser you get for any vault.

And the badge says what the key allows. One read, zero writes.

pip install sgit dash ai. Or try it in your browser, at sgit dot ai.

This video was made by an agent, end to end, with no human at a screen and no API cost. The numbers are on the screen.

Renders

CUT	LENGTH	SIZE	SLIDES	TTS	RECORD	DRIFT	PIPELINE
landscape 1280×720 · Kokoro	02:05.75	9.4 MB	15	166 s	125.9 s	176 ms	302 s
shorts 1080×1920 · Kokoro	01:09.94	6.5 MB	8	96 s	70.0 s	127 ms	177 s
landscape 1280×720 · OpenRouter gpt-audio	01:50.55	8.0 MB	15	5 s	110.6 s	160 ms	127 s

Source of truth is `reel.json`. Stills in `images/` (desktop) and `images-shorts/` (phone); clips in `clips/`. Timecodes are the landscape cut's TTS durations.